FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

SMART FLOW COMMUTE
INTELLIGENT URBAN COMMUTING ASSISTANT

A PROJECT REPORT

Submitted by

SARVESH R - 240701478

RITTICK V - 240701432

III YEAR

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI – 602 105



OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that the project titled “Smart Flow Commute: Intelligent Urban Commuting Assistant” is the project work carried out by Sarvesh R and Rittick V under the guidance of Mrs. Deepa, in partial fulfilment of the requirements of the course Foundations of Artificial Intelligence.

This report documents the motivation, conceptual design, proposed workflow, implementation-review approach, testing plan, risks, and future development of the Smart Flow Commute web application. The project website is hosted at https://smart-flow-commute.lovable.app/. Since the live page could not be independently inspected in detail while preparing this report, statements about specific deployed functions, data integrations, measured performance, and internal technologies are deliberately treated as items to verify against the running application and source code.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on: _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	04
	LIST OF TABLES	05
	LIST OF FIGURES	05
	LIST OF SYMBOLS AND ABBREVIATIONS	05
1	INTRODUCTION	06
1.1	Problem Statement and Motivation	06
1.2	Objectives and Scope	06
2	LITERATURE REVIEW	07
3	DOMAIN AREA AND BACKGROUND	08
4	PROBLEM FORMULATION	09
5	PROPOSED SYSTEM	10
6	SYSTEM ARCHITECTURE AND DESIGN	12
7	ROUTE PLANNING AND INTELLIGENT DECISION METHODS	14
8	IMPLEMENTATION DETAILS	16
9	TESTING AND VALIDATION	19
10	EXPERIMENTAL EVALUATION	20
11	RESULTS AND DISCUSSION	22
12	LIMITATIONS AND RISK ANALYSIS	23
13	FUTURE WORK	24
14	LEARNING OUTCOMES	25
15	CONCLUSION	26
16	ACKNOWLEDGEMENT	27
17	REFERENCES	28

Note: Page numbers are indicative and should be refreshed after final editing or pagination in Microsoft Word.

ABSTRACT

Daily commuting in urban areas is affected by congestion, variable travel times, road incidents, weather, construction, and incomplete or delayed information. A route that is normally convenient may become inefficient when conditions change. Smart Flow Commute is a web-based project concept intended to support more informed commuting by presenting journey-related information and helping users reason about route choices. This report frames the project as an intelligent urban commuting assistant and develops a structured account of its problem domain, requirements, system design, candidate routing methods, evaluation criteria, and responsible deployment considerations.

The report uses a layered design model comprising a user interface, trip and road-network representation, information collection layer, route evaluation component, and result presentation layer. Graph-based routing is discussed as a suitable foundation: intersections can be represented as vertices, road segments as edges, and edge costs can represent travel time or other configurable factors. Dynamic factors such as incidents or weather may be incorporated only when reliable data sources and appropriate update mechanisms are available. The design distinguishes features that are conceptually proposed from features that must be confirmed in the deployed application.

Because the live website could not be fully inspected during report preparation and no source repository or execution logs were supplied, this document does not claim verified live traffic feeds, GPS tracking, AI prediction accuracy, or measured time savings. Instead, it defines a repeatable testing and evaluation protocol, identifies evidence that should be collected from the actual application, and records implementation-specific details as verification items. The project illustrates how intelligent routing principles can be organized into a practical web application while emphasizing data quality, transparent recommendations, privacy, accessibility, and independent validation.

Keywords—Smart commuting, urban mobility, route planning, graph search, travel-time estimation, web application, traffic information, decision support, validation.

LIST OF TABLES

Table No.	Title	Page No.
1	Functional requirements	18
2	Non-functional requirements	18
3	Candidate route-planning methods	22
4	Conceptual data entities	25
5	Technology and evidence register	26
6	Test case summary	29
7	Experimental data collection template	32
8	Risk analysis register	37
9	Requirement-to-test traceability	31

LIST OF FIGURES

Figure No.	Title	Page No.
1	Conceptual commuting decision flow	11
2	Proposed layered architecture	17
3	Trip-planning workflow	19
4	Road network as a weighted graph	21
5	Route evaluation and re-planning loop	23
6	Evidence capture checklist	27

LIST OF SYMBOLS AND ABBREVIATIONS

Term	Full form / description
AI	Artificial Intelligence
API	Application Programming Interface
ETA	Estimated Time of Arrival
GPS	Global Positioning System
GIS	Geographic Information System
UI	User Interface
UX	User Experience
URL	Uniform Resource Locator
HTTP	Hypertext Transfer Protocol
G=(V,E)	Graph with vertex set V and edge set E
w(e)	Cost assigned to road edge e
P	Candidate path from origin to destination
C(P)	Total cost of path P
ETA	Estimated arrival time

1. INTRODUCTION

1.1 Problem Statement and Motivation

Commuters often choose routes using habitual knowledge, static maps, or information that may not reflect current road conditions. Travel time can vary because of peak-hour demand, traffic incidents, road works, weather, public events, and differences in road capacity. The practical problem is not simply finding a geographically short path; it is selecting a route that is reasonable for the user's current journey and the information available at decision time.

Smart Flow Commute is positioned as a web-based commuting assistant intended to make journey planning clearer and more responsive. Its value depends on presenting understandable options and on the quality, freshness, and coverage of any travel information it uses. A recommendation should not be treated as guaranteed truth: the system should communicate uncertainty when inputs are incomplete or stale.

1.2 Objectives and Scope

- Define the commuting problem in terms of origin, destination, candidate paths, travel costs, and constraints.
- Describe a modular web architecture that can accept trip information and present route guidance.
- Identify graph search and route-cost evaluation methods suitable for a first implementation.
- Specify functional and non-functional requirements that can be tested independently.
- Establish a repeatable validation plan for correctness, response time, usability, and robustness.
- Document limitations and avoid claims of live data, predictive accuracy, or savings without evidence.

The report covers the web application as a prototype or project-level solution. Deployment scale, integrations, commercial operation, and the availability of live location or traffic feeds must be verified separately. The project should be evaluated against the behavior actually implemented rather than against every capability that a future smart-mobility platform might support.

This report documents the motivation, conceptual design, proposed workflow, implementation-review approach, testing plan, risks, and future development of the Smart Flow Commute web application. The project website is hosted at https://smart-flow-commute.lovable.app/. Since the live page could not be independently inspected in detail while preparing this report, statements about specific deployed functions, data integrations, measured performance, and internal technologies are deliberately treated as items to verify against the running application and source code.

2. LITERATURE REVIEW

2.1 Graph-Based Route Planning

Road networks are commonly represented as graphs. Intersections or road junctions form vertices, and road segments form edges. Each edge can be assigned a cost such as distance, expected travel time, toll, or a weighted combination of user preferences. A path is a sequence of connected edges from origin to destination. This representation provides a precise foundation for route search and enables the cost of alternative paths to be compared.

2.2 Shortest-Path Algorithms

Dijkstra's algorithm finds a minimum-cost path in a graph with non-negative edge weights. A* search uses a heuristic estimate of the remaining cost to guide exploration and can reduce search effort when the heuristic is appropriate. Breadth-first search is suitable for unweighted graphs but does not account for varying travel times unless each transition has equal cost. The algorithm should be selected according to the graph representation, cost definition, and performance needs.

2.3 Dynamic Routing and Traffic Information

Static routing assumes that edge costs remain fixed during a search. Real commuting conditions are dynamic: congestion or an incident may change the cost of a road after a route has been calculated. Dynamic routing therefore requires an update policy, timestamped observations, and a rule for deciding when a new route should be computed. A system must avoid repeatedly changing routes because of small or noisy cost fluctuations.

2.4 Decision Support and User Trust

Route recommendations are more useful when they explain the factors behind a choice, such as expected duration, distance, tolls, or a reported disruption. Users need to understand whether a value is measured, estimated, or inferred. A clean interface and a fast response are insufficient if the underlying data is unreliable or the system hides uncertainty.

2.5 Project Positioning

The project is an applied commuting-support application rather than a claim of a new routing algorithm. Its contribution is the organization of trip input, route evaluation, result presentation, and future validation into a single user-facing workflow. Any claim of real-time intelligence, AI prediction, or sensor integration should be supported by the deployed implementation and a traceable data source.

3. DOMAIN AREA AND BACKGROUND

3.1 Urban Mobility

Urban mobility includes the movement of people through a network of roads, paths, public transport links, and interchange points. Commuting decisions are influenced by travel time, reliability, cost, safety, accessibility, and personal constraints. A route that minimizes distance may not minimize time, and a route that minimizes expected time may not be suitable for every user.

3.2 Travel Network Representation

Let the road network be represented by G=(V,E), where V is the set of junctions or mapped locations and E is the set of traversable road segments. Each edge e has attributes such as length, permitted direction, estimated speed, and optional incident or weather status. A path P is valid when consecutive edges connect and all traversal restrictions are satisfied.

3.3 Route Cost

A basic route cost can be the sum of edge travel-time estimates. More advanced formulations can incorporate tolls, transfers, road restrictions, or user-selected preferences. When several factors are combined, weights must be documented and normalized so that the score has a meaningful interpretation. A numerical score should not be presented as an objective measure of safety or quality unless its definition and data sources justify that interpretation.

3.4 Static and Dynamic Inputs

Static inputs include the road-network structure, road classifications, and known restrictions. Dynamic inputs may include traffic speeds, closures, weather, or incident reports. Dynamic data should carry a source, timestamp, and confidence or quality indicator. When no live feed is connected, the application should clearly describe its output as based on static or demonstration data.

3.5 User Roles and Context

A basic commuting assistant may be used by a commuter planning a trip. Administrative or operational roles would be relevant only if the deployed application includes data management or reporting functions. The current report does not assume account registration, administrator dashboards, stored user profiles, or a database unless these are verified in the application.

4. PROBLEM FORMULATION

4.1 Formal Definition

Given an origin o, destination d, a road graph G=(V,E), and a non-negative cost function w(e), find a feasible path P from o to d that minimizes the total path cost $C(P)=\Sigma\, w(e)$ for all edges e in P. Feasibility also depends on directionality, road access restrictions, closures, and any user constraints represented by the system.

4.2 Inputs and Outputs

Element	Description	Verification question
Origin	Starting location selected or entered by the user	Does the interface validate and resolve it?
Destination	Target location for the journey	Can invalid or ambiguous destinations be handled?
Road graph	Locations and connecting road segments	What map or network source is actually used?
Edge cost	Distance, time, or composite cost	Is the cost measured, estimated, or illustrative?
Constraints	Road restrictions and user preferences	Which constraints are encoded in the current build?
Output	Route option(s), summary, and any guidance	Does the output match the underlying route result?

4.3 Hard Constraints and Preferences

Hard constraints determine whether a path is permissible. Examples include traversing only connected edges, respecting one-way roads, and excluding confirmed closures when closure data is available. Preferences influence ranking rather than feasibility, such as preferring shorter journeys or avoiding tolls. A preference should not override a safety-critical or legal restriction.

4.4 Objective Function

For a travel-time objective, $C(P)=\Sigma\, t(e)$, where t(e) is the estimated time to traverse edge e. For a multi-criteria objective, one possible form is $C(P)=\alpha T(P)+\beta D(P)+\gamma R(P)$, where T is travel time, D is distance, R is a defined route penalty, and α, β, γ are documented weights. This is a design option, not a claim about the current website's implementation.

4.5 Assumptions

- The origin and destination can be represented within the chosen road network.
- Every traversable edge has a valid, non-negative cost for the selected algorithm.
- Any external data source, if used, is available and its update frequency is understood.
- Demonstration or synthetic data is labelled as such and is not presented as live observation.
- Evaluation measurements are collected from actual runs and preserved with configuration details.

4.6 Failure Conditions

A route search may fail because the input is invalid, the destination is unreachable, the road graph is incomplete, a required service is unavailable, or an internal error occurs. These cases should not be collapsed into a generic message. If no route is returned, the application should communicate the reason when it can be determined and allow the user to revise the request.

5. PROPOSED SYSTEM

5.1 System Overview

The proposed Smart Flow Commute workflow accepts trip information, prepares a route-planning request, evaluates feasible paths, and presents the resulting journey information. Where dynamic traffic or incident data is available, an update component can refresh relevant edge costs and trigger re-evaluation. The live application's exact behavior must be confirmed by testing the deployed site.

5.2 Main Logical Modules

Module	Responsibility	Status in this report
Trip input	Collect origin, destination, and optional preferences	Required concept; verify exact controls
Input validation	Check missing, malformed, or unsupported trip values	Recommended requirement
Network representation	Provide connected locations and road segments	Data source must be verified
Route engine	Compute a feasible path using a selected algorithm	Candidate design
Cost evaluator	Calculate time, distance, or preference score	Depends on available data
Result presentation	Show route and journey summary	Verify deployed output
Refresh/re-plan	Update route after relevant conditions change	Future or integration-dependent
Error handling	Explain invalid inputs, unavailable data, or no path	Recommended requirement

5.3 User Workflow

1. Open the deployed Smart Flow Commute URL and identify the available trip-planning controls.
2. Enter or select an origin and destination using the interface actually provided.
3. Submit the request and allow the application to validate the input.
4. Review the displayed route, estimated journey information, and any stated assumptions.
5. If alternatives are offered, compare them using the displayed cost criteria.
6. Record errors, unavailable services, or inconsistent results during testing.

5.4 Expected Outputs

Expected outputs may include a selected route, route alternatives, a journey summary, and a status message. These are design expectations and must be reconciled with the live application. The report intentionally does not assert that turn-by-turn navigation, GPS tracking, live congestion, weather alerts, or saved trip history are implemented.

5.5 Error Handling and Usability

A useful interface should distinguish invalid input from a valid request for which no route can be found. It should provide visible loading feedback for long requests, prevent accidental duplicate submissions, and make errors actionable. If external services fail, the user should not be shown stale results without a timestamp or warning.

5.6 Conceptual Workflow Figure

Step	Process	Output
1	Enter trip request	Origin, destination, preferences
2	Validate request	Valid request or clear error
3	Resolve network and costs	Graph and edge weights
4	Search candidate paths	Feasible path or no-path status
5	Evaluate and rank	Route cost and comparison

6	Present result	Route summary and caveats
7	Optional refresh	Updated costs and re-planning

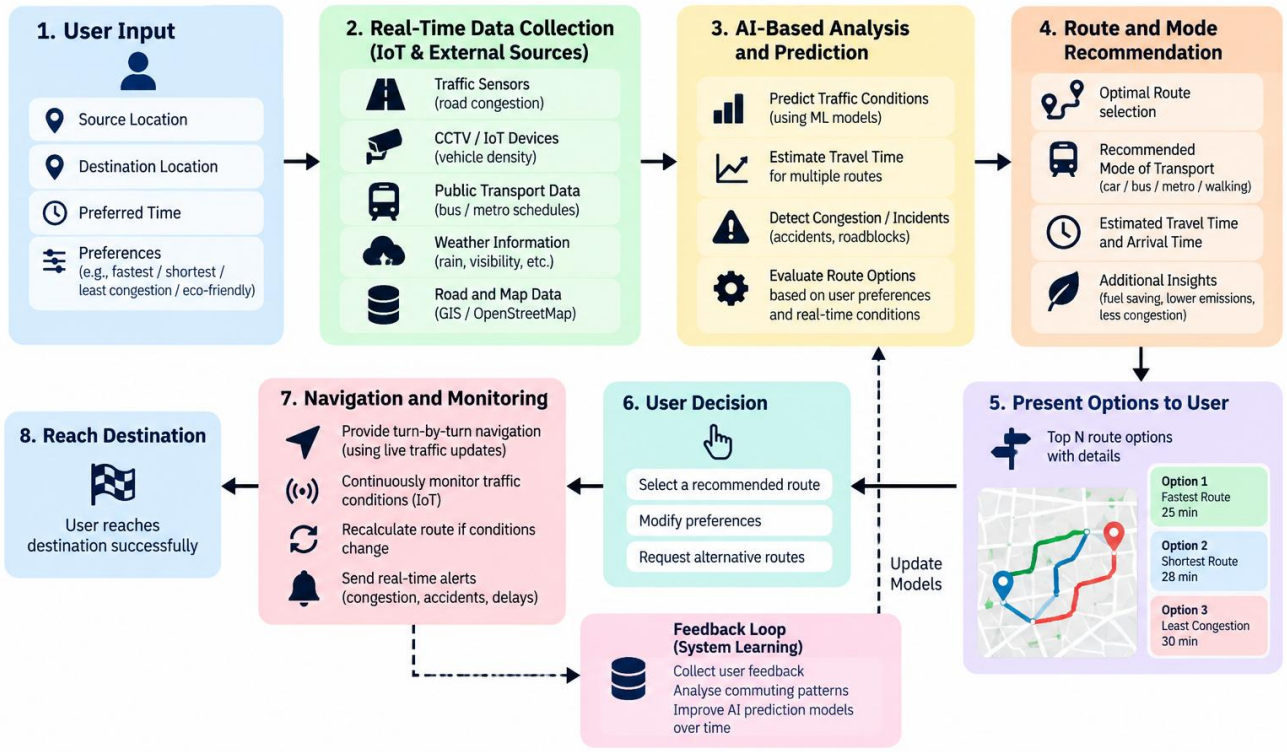


FIGURE 1: Conceptual commuting decision flow

6. SYSTEM ARCHITECTURE AND DESIGN

6.1 Architecture Overview

A maintainable commuting assistant can be organized into presentation, request handling, route-planning, data-access, and validation components. The presentation layer handles user interaction; request handling validates input; route planning operates on a graph and cost model; data access obtains road or traffic information; and validation checks that the returned path is connected and complies with the encoded restrictions. This logical architecture should be updated to match the actual source code and integrations.

Layer	Core responsibility	Design consideration
Presentation	Trip form, status, route summary	Readable labels, responsive layout, accessible feedback
Request handling	Normalize and validate input	Reject malformed coordinates and missing fields
Routing service	Search the graph for a feasible path	Algorithm appropriate to graph costs
Data adapter	Load road graph and optional dynamic inputs	Track source, freshness, and failures
Result validator	Check path continuity and constraints	Independent checks reduce shared defects
Observability	Record errors and timing for evaluation	Avoid logging unnecessary personal location data

6.2 Presentation Layer

The user interface should keep the primary journey task visible and reduce ambiguity. If the app displays a map, route line, estimated time, distance, or alternative paths, each item should be labelled clearly. Colors should not be the only way to distinguish route status, and mobile layouts should remain usable without horizontal scrolling.

6.3 Routing Engine

The routing engine receives a normalized origin and destination, identifies corresponding graph nodes, searches candidate paths, and returns a structured result. Its interface should separate graph search from UI rendering so that the same algorithm can be tested using small, known graphs. The engine should return explicit statuses for success, invalid input, unreachable destination, and internal failure.

6.4 Data and Integration Layer

Any map, geocoding, traffic, or weather integration should be documented with its provider, permitted use, rate limits, update frequency, and failure behavior. If the application currently uses only a visual prototype or fixed sample data, that limitation must be stated. A map visible in the browser does not by itself prove that live traffic or predictive intelligence is connected.

6.5 Conceptual Architecture Diagram

User / Browser	Application Logic	Data / Services
Trip form and route display	Input validation and request coordination	Road-network or map provider
Route summary and status	Graph search and cost evaluation	Optional traffic / incident source
Error feedback	Independent route validation	Logs and evaluation records

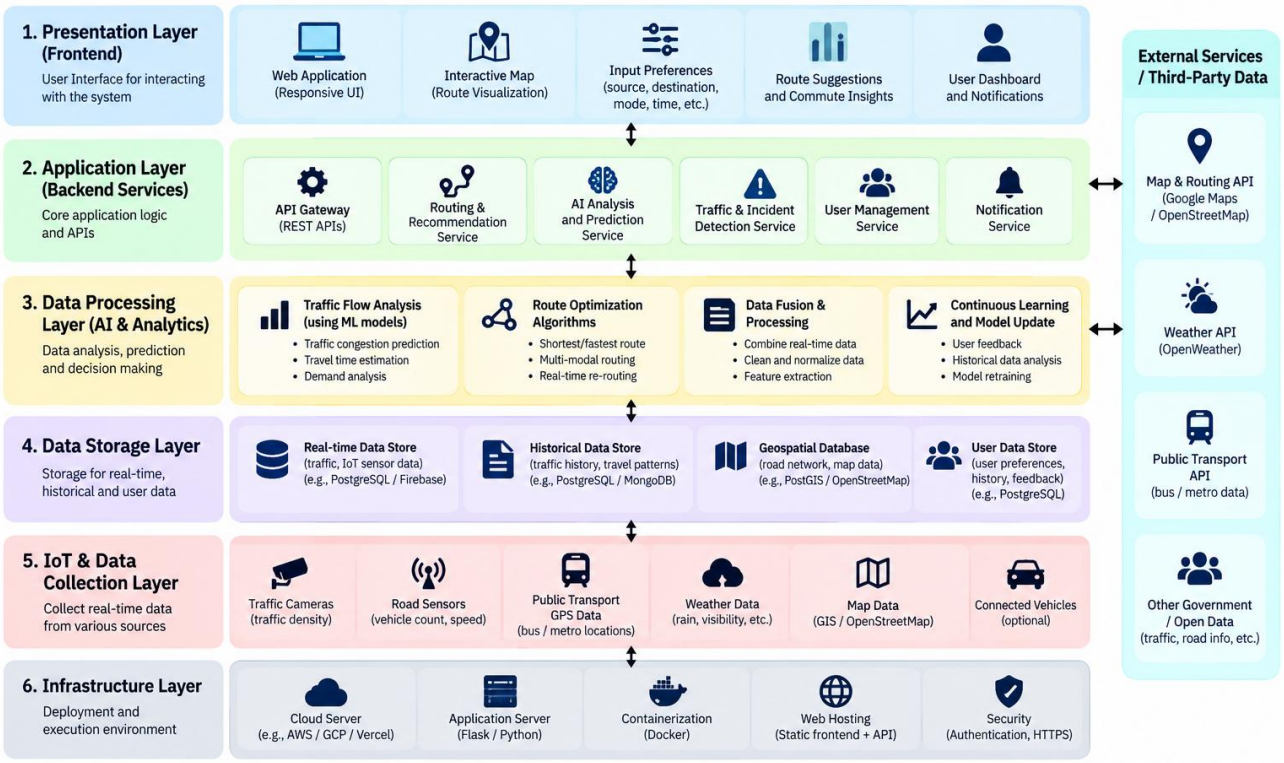


FIGURE 2: Proposed layered architecture for Smart Flow Commute.

6.6 Security, Privacy, and Reliability

Origin and destination can reveal personal routines. The application should request only the location information needed for the trip, explain any location permission, and avoid retaining trip history unless there is a clear purpose and user notice. External requests should use secure transport, credentials should not be embedded in public client code when they must remain secret, and service failures should degrade gracefully.

7. ROUTE PLANNING AND INTELLIGENT DECISION METHODS

7.1 Breadth-First Search

Breadth-first search explores nodes level by level and finds a path with the fewest edges in an unweighted graph. It is useful for teaching and small network demonstrations but is not sufficient for realistic travel-time optimization when road segments have different costs.

7.2 Dijkstra's Algorithm

Dijkstra's algorithm repeatedly selects the unsettled node with the lowest known accumulated cost and relaxes its outgoing edges. With non-negative edge weights, it computes a minimum-cost path. For commuting, the edge weight may represent estimated travel time or distance. Its result is only as reliable as the graph and edge costs supplied to it.

7.3 A* Search

A* search ranks a node using $f(n)=g(n)+h(n)$, where $g(n)$ is the known cost from the origin and $h(n)$ estimates the remaining cost to the destination. A suitable heuristic can guide search toward the destination. For a time-based objective, a distance heuristic must be converted consistently and should not overestimate if optimality is required under standard A* conditions.

7.4 Dynamic Edge Costs

When reliable traffic observations are available, edge costs may be updated to reflect current conditions. A simple model estimates travel time as segment length divided by estimated speed. The system should specify how missing observations are handled, how old data becomes stale, and how often a route is recomputed. Dynamic costs introduce uncertainty and do not guarantee that the predicted arrival time will match the actual trip.

7.5 Route Ranking and Trade-Offs

A route planner may compare alternatives by travel time, distance, tolls, or other explicit preferences. These dimensions can conflict. The interface should avoid hiding a long detour behind a single score and should explain the meaning of any ranking. Safety-related restrictions should be handled as feasibility rules rather than ordinary preferences.

7.6 Algorithm Comparison

Method	Best use	Strength	Limitation
BFS	Unweighted graphs	Simple and complete for reachable nodes	Ignores varying edge costs
Dijkstra	Non-negative weighted graph	Finds minimum-cost path	May explore many nodes
A*	Weighted graph with useful heuristic	Can reduce exploration	Depends on heuristic validity
Dynamic re-planning	Changing edge costs	Can react to updated information	Requires fresh, reliable data

7.7 High-Level Pseudocode

ROUTE(G, origin, destination):

1. Validate origin and destination; return a clear error if invalid.

2. Resolve both locations to nodes in graph G.

3. Initialize the search with origin cost zero.

4. Repeatedly select the next node according to the chosen search algorithm.

5. Relax or explore connected edges while respecting graph restrictions.

6. If destination is reached, reconstruct the path and calculate its summary.

7. Independently verify path connectivity and return the result.

8. If no path exists, return an explicit no-route status.

7.8 Complexity and Practical Performance

For a graph with |V| vertices and |E| edges, Dijkstra's algorithm with a binary heap is commonly described as O((|V|+|E|) log |V|). A* has graph-dependent practical behavior and can approach broad exploration when the heuristic is weak. Real application latency also includes geocoding, network calls, rendering, and data loading; these should be measured separately from search time.

8. IMPLEMENTATION DETAILS

8.1 Development and Deployment Context

The supplied project URL is https://smart-flow-commute.lovable.app/. The URL indicates a web-hosted application created or published through Lovable, but it does not establish the complete runtime stack or the details of any connected backend. The source repository, dependency manifest, database schema, and provider configuration were not supplied for this report. Those details must be extracted from the actual project before a final technical submission.

8.2 Interface and Request Flow

The implementation review should identify the page components, input controls, event handlers, request functions, and output components involved in one trip-planning action. For each control, record its validation rules and the exact response shown for success, failure, or empty data. The browser interface should not be considered proof that route calculation or live data is functioning correctly.

8.3 Conceptual Data Entities

Entity	Representative fields	Purpose
TripRequest	origin, destination, preferences, request time	Represents one planning request
Location	identifier, label, coordinates if supported	Represents a resolved point
RoadSegment	start node, end node, distance, cost	Represents a traversable connection
RouteResult	path, cost summary, status, timestamp	Represents a returned route
ConditionRecord	source, value, observed time, confidence	Optional dynamic input if integrated
TestRun	version, scenario, duration, result, notes	Supports repeatable evaluation

These are conceptual entities for documentation and test planning. They are not a claim that the application currently stores these records or uses a relational database.

8.4 Technology Stack Evidence Register

Technology / component	Known from available evidence	Required verification
Lovable hosting / project URL	The provided URL uses the lovable.app domain	Confirm deployment settings and source export
Frontend framework	Not independently verified	Inspect package manifest and source files
Map or geocoding provider	Not verified	Inspect imports, environment variables, and network requests
Routing algorithm	Not verified	Trace route calculation in source code
Backend / database	Not verified	Check integrations and server configuration
Authentication / persistence	Not verified	Test actual flows; do not assume they exist

8.5 Input Validation

The application should validate required fields, normalize text or coordinates, and handle ambiguous or unsupported locations. If the user enters a place name, the system should report when it cannot resolve the location. If coordinates are accepted, values should be checked for valid numeric ranges. Invalid input should be rejected before expensive route computation.

8.6 Error Handling and Logging

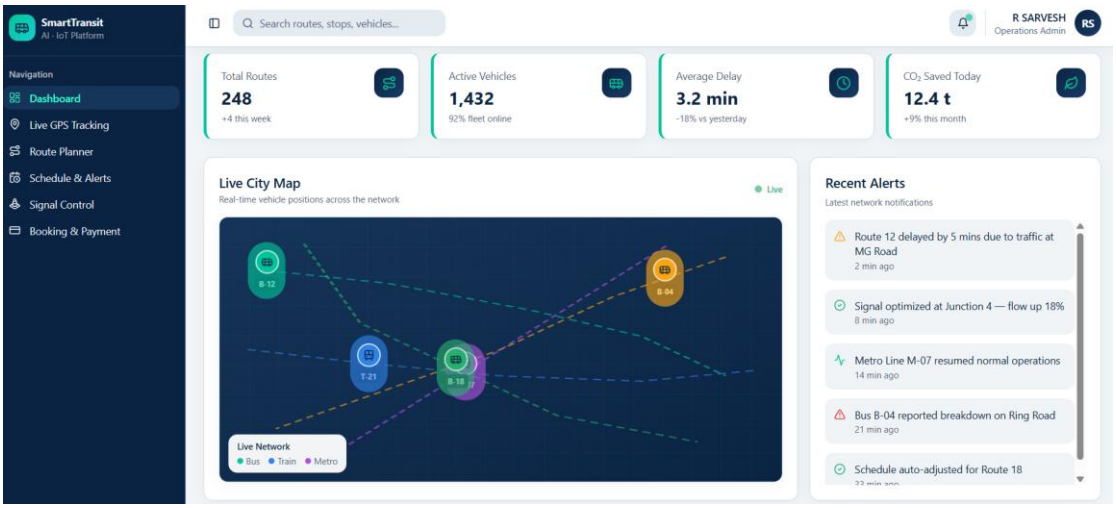
Potential errors include unavailable map services, rate limits, network failures, malformed responses, unreachable destinations, and rendering failures. Error handling should preserve a clear user message while

recording enough diagnostic context for developers. Logs should avoid retaining exact location histories unless that retention is necessary, disclosed, and protected.

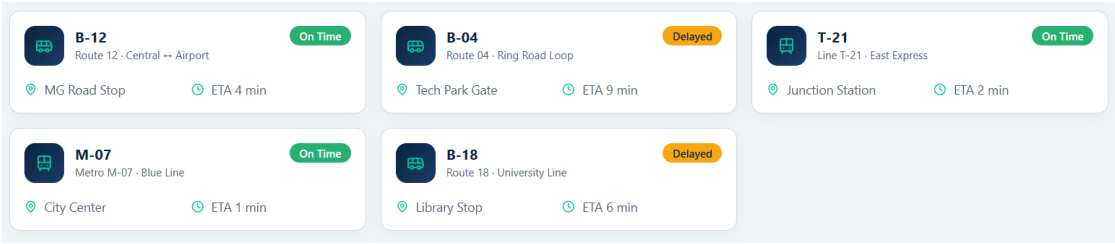
8.7 Screenshot Evidence Register

Insert screenshots captured from the actual deployed application in the following places. Do not use generated mockups as evidence of implemented behavior.

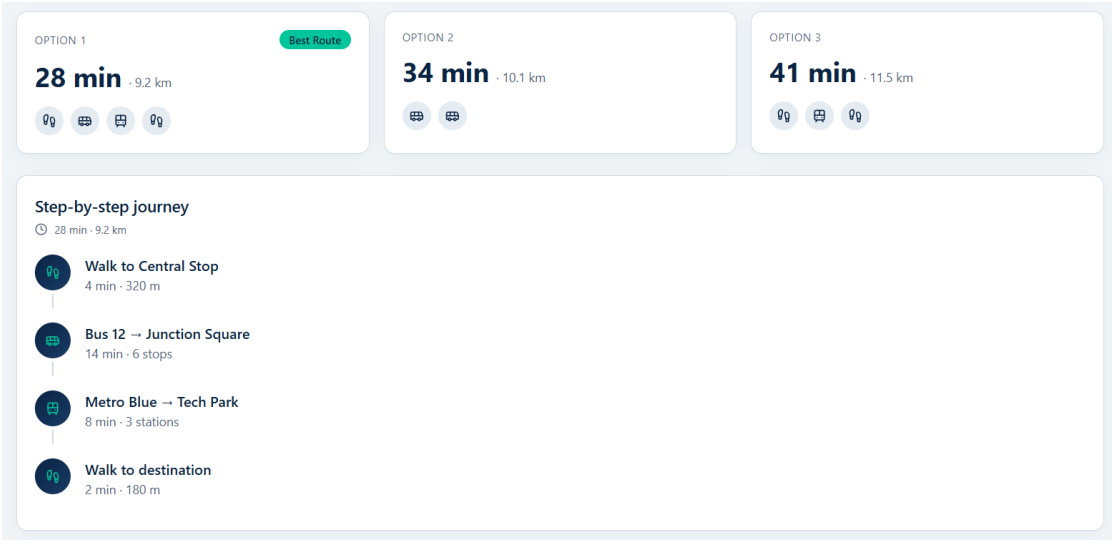
- Figure A1 — landing page or main commuting screen, showing the actual page title and available controls.



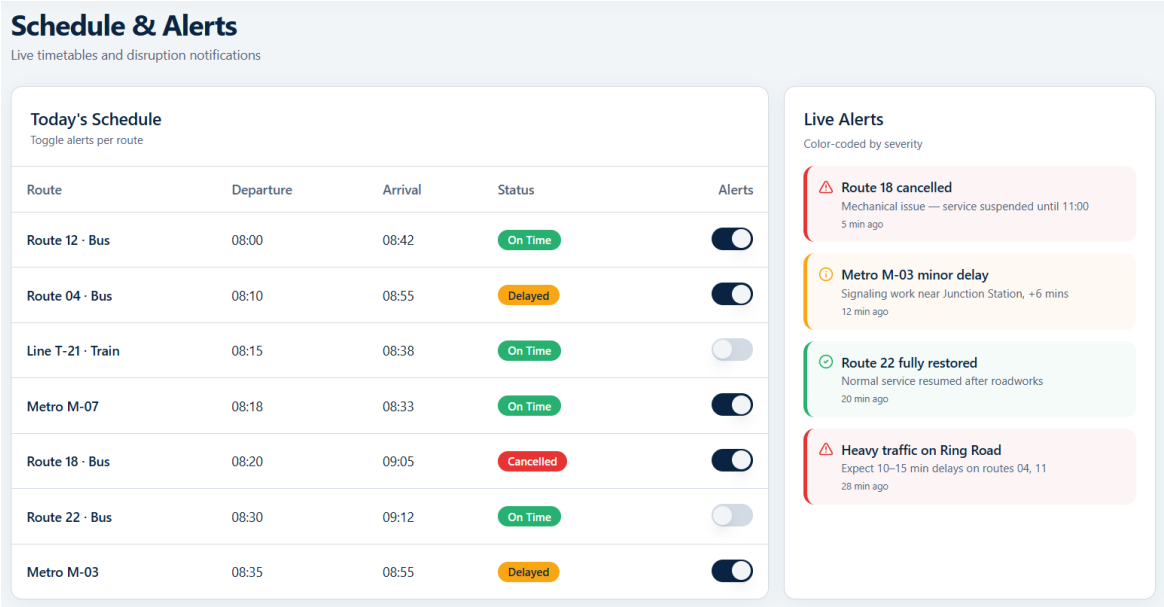
- Figure A2 — trip input with a valid origin and destination.



- Figure A3 — successful route result with all visible summary fields.



- Figure A4 — invalid or unresolved location and the resulting error message.



8.8 Reproducibility

For each test run, record the URL and build/version, date and time, browser, device viewport, input scenario, data source state, and observed output. If the project can be exported from Lovable to a repository, record the commit identifier. Repeat tests with the same input to detect nondeterministic output or dependence on changing external data.

9. TESTING AND VALIDATION

9.1 Testing Objectives

Testing should establish that trip input is validated, the route engine returns connected paths, displayed route details match the result, and errors are communicated correctly. If live traffic or location integration is claimed, additional tests must establish the source, freshness, permission behavior, and failure modes of those integrations.

9.2 Unit Testing

Unit tests should target location validation, graph connectivity, edge-cost calculation, path reconstruction, and error classification. Small graphs with known optimal paths are especially useful because the expected answer can be calculated independently. A route result should be checked for continuity from origin to destination and for compliance with the encoded restrictions.

9.3 Integration Testing

Integration tests should verify that the interface sends the expected request, the application receives or calculates the result, and the UI renders the returned status and route details. When a provider is unavailable, the application should not silently substitute fabricated values. Test both successful and failed service responses.

9.4 Test Case Summary

Test ID	Scenario	Expected behavior	Status
TC-01	Valid origin and destination	A valid result or documented no-route status	Pending execution
TC-02	Missing origin	Clear validation message	Pending execution
TC-03	Missing destination	Clear validation message	Pending execution
TC-04	Unknown location	Unresolved location handled explicitly	Pending execution
TC-05	Disconnected graph	No-route status, no invalid path	Pending execution
TC-06	Known weighted graph	Returned path has minimum expected cost	Pending execution
TC-07	Malformed provider response	Graceful error, no crash	Pending execution
TC-08	Repeated identical request	Consistent output when inputs/data are fixed	Pending execution
TC-09	Narrow/mobile viewport	Controls remain usable	Pending execution
TC-10	External service unavailable	Clear failure or documented fallback	Pending execution
TC-11	Path continuity	Every consecutive edge connects	Pending execution
TC-12	Stale dynamic data, if used	Timestamp/freshness policy applied	Not applicable until verified

9.5 Validation Evidence

For each test, preserve the input, expected behavior, actual behavior, pass/fail status, and evidence. A screenshot can demonstrate what the interface displayed but does not independently prove the optimality of a route. Where feasible, compare the application result with a trusted reference calculation on a small graph or with a documented provider result.

9.6 Extended Testing

- Boundary tests for very short journeys and long-distance journeys supported by the application.
- Tests with one-way edges, closed edges, and disconnected nodes when those rules exist in the model.
- Regression tests after changes to map integration, route costs, or input normalization.
- Performance tests at increasing graph sizes using a fixed dataset.
- Accessibility checks for labels, keyboard operation, text contrast, and screen-reader interpretation.

- Privacy checks confirming that location permissions and any storage behavior match the interface notice.

9.7 Requirement-to-Test Traceability

Requirement	Evidence / test	Status
Accept valid trip input	TC-01 and screenshot A2	Pending
Reject invalid input	TC-02 to TC-04	Pending
Return a connected route	TC-05, TC-06, TC-11	Pending
Show understandable output	TC-01 and screenshot A3	Pending
Handle service failure	TC-07 and TC-10	Pending
Support narrow view	TC-09 and screenshot A5	Pending
Use fresh dynamic data if claimed	TC-12 plus provider timestamps	Verify whether applicable

9.8 User Interface Evaluation

A short usability evaluation can ask participants to plan a sample journey, interpret the result, compare any alternatives, and recover from an invalid input. Record task completion, points of confusion, and comments. Do not report user satisfaction or usability scores unless participants actually completed the evaluation.

10. EXPERIMENTAL EVALUATION

10.1 Evaluation Questions

- RQ1: Does the route engine return a valid connected path for supported inputs?
- RQ2: Does the returned path minimize the documented cost on known test graphs?
- RQ3: How does route-computation time change as graph size increases?
- RQ4: How clearly does the interface communicate invalid input and unavailable data?
- RQ5: If dynamic data is integrated, how quickly and correctly does the route respond to changed edge costs?

10.2 Experimental Setup

Use a small set of manually verified graphs for correctness tests and larger synthetic graphs for performance tests. Each experiment should record the number of vertices and edges, origin and destination, edge-cost distribution, algorithm and configuration, software version, hardware/browser context, and timing boundary. Separate routing time from geocoding, external requests, and UI rendering.

10.3 Metrics

Metric	Definition	Collection method
Path validity	Whether the path is connected and respects encoded restrictions	Independent validator
Path cost	Sum of the documented edge costs	Recalculate from returned path
Optimality gap	Returned cost compared with known optimal cost	Known-answer graph or trusted solver
Search duration	Time spent in route computation	Instrument route engine
End-to-end duration	User submission to visible result	Browser-level timing
Failure rate	Failed requests divided by total test requests	Test log
Freshness	Age of dynamic data at decision time, if used	Source timestamps

10.4 Experimental Data Template

Run ID	Graph size (V/E)	Scenario	Duration	Validity / cost	Status
R-01	[record]	Known optimal path	[measure]	[verify]	Not run
R-02	[record]	Disconnected destination	[measure]	N/A	Not run
R-03	[record]	Larger graph	[measure]	[verify]	Not run
R-04	[record]	Service failure, if applicable	[measure]	N/A	Not run

10.5 Fair Comparison

When comparing algorithms, use identical graph inputs, edge costs, and stopping conditions. Repeat performance runs and report the aggregation method, such as median and range. Record environment details and avoid comparing browser-rendering time for one configuration with solver-only time for another. No performance numbers are supplied in this report because no validated measurements were provided.

10.6 Interpretation

A route being displayed does not prove that it is optimal. Correctness requires checking path continuity and costs against a known reference. A fast response does not prove accurate traffic prediction. If the application uses dynamic estimates, prediction error should be measured against observed outcomes over a defined time window and geography.

11. RESULTS AND DISCUSSION

11.1 Current Evidence Status

The project URL was supplied, but the live page could not be inspected in sufficient detail during report preparation. No source code, test logs, screenshots, dataset, or benchmark results were supplied. Consequently, this report describes a system design and an evaluation plan rather than claiming verified algorithm behavior or measured performance.

Evidence item	Current status	Action before submission
Project URL	Provided by project owner	Open and capture current deployed build
Implemented features	Not independently verified	Map each visible feature to source/code
Technology stack	Only hosting URL is known	Inspect exported source and package files
Routing correctness	No test output supplied	Run known-answer graph tests
Performance measurements	Not supplied	Collect repeatable timing data
Screenshots	Not supplied	Insert genuine screenshots with captions

11.2 Expected Algorithmic Behavior

For a valid connected graph with non-negative edge weights, an appropriate shortest-path algorithm should return a connected path whose total cost is no greater than any alternative path in the tested graph. If the destination is disconnected, the solver should return a no-path status. These are expected properties of the proposed design and not results observed from the current website.

11.3 Practical Discussion

The main engineering challenge is often not the graph-search algorithm alone but the reliability of the road network, location resolution, and cost estimates. A route planner can be mathematically correct while still recommending a poor journey if its data is stale or incomplete. The system should therefore communicate data provenance and uncertainty and should separate algorithm correctness from real-world travel-time accuracy.

11.4 Result Tables and Figures

After tests are executed, replace the placeholders in Chapter 10 with actual values and include at least one comparison table, a plot of runtime against graph size, and screenshots of successful and failed requests. Every figure should identify the test scenario and application version. Do not add fabricated charts or illustrative numbers as if they were measured results.

11.5 Summary

The project provides a useful setting for applying graph-based search and web-interface design to everyday commuting. Its practical quality will depend on validating the deployed behavior, clarifying which data sources are used, and testing the route result independently. The next stage should prioritize evidence collection over unsupported claims.

12. LIMITATIONS AND RISK ANALYSIS

12.1 Data Availability and Freshness

A route planner depends on the completeness and freshness of its road-network and traffic inputs. If the app uses static or demonstration data, it cannot reliably represent changing congestion. If it depends on a third-party service, availability, coverage, licensing, quotas, and latency become external risks.

12.2 Model Completeness

A model may omit one-way restrictions, road closures, access rules, turn restrictions, pedestrian limitations, or mode-specific constraints. Such omissions can produce paths that are connected in the simplified graph but not usable in the real world. The supported travel modes and constraints must be documented explicitly.

12.3 Estimation and Safety Risks

Estimated arrival times are uncertain. Users should not be encouraged to treat an estimate as a guarantee or to interact with the interface while driving. The application should direct users to plan before departure and follow local traffic laws and road signage.

12.4 Privacy and Security Risks

Location data can expose home, workplace, and routine travel patterns. Data collection should be minimized, transmission secured, and retention explained. API keys and privileged credentials should be protected. Any analytics or user accounts should be assessed before the app is used with real personal data.

12.5 Risk Register

Risk	Likelihood / impact	Mitigation	Evidence to monitor
Stale road or traffic data	Medium / High	Timestamp data and define stale-data policy	Source timestamp and coverage
Invalid location resolution	Medium / Medium	Validate and show correction options	Failed-resolution tests
Route violates road rule	Low–Medium / High	Encode restrictions and independently validate	Constraint test suite
External service outage	Medium / High	Graceful failure and clear status	Failure-injection tests
Misleading ETA	Medium / High	Label estimates and measure error	Observed vs estimated times
Location privacy exposure	Medium / High	Minimize retention and restrict access	Privacy review and storage audit
Unsupported feature claims	Medium / Medium	Maintain feature-evidence register	Code and UI verification

12.6 Ethical and Operational Boundaries

The system should not present speculative traffic or safety scores as verified facts. Data sources should be used lawfully and with respect to provider terms. If a route recommendation is based on incomplete information, that limitation should be visible. Accessibility needs and differing travel modes should be considered before expanding the product's claims.

12.7 Limitations of This Report

This report was prepared without access to the application's source repository or validated execution evidence. The live site could not be inspected in detail through the available browsing session. Therefore, architecture and data entities are conceptual, tests are pending, and numerical results are intentionally omitted. Before academic submission, the team should verify every implementation-specific statement and replace conceptual figures with evidence from the real application.

13. FUTURE WORK

13.1 Verify and Document the Current Build

The first priority is to capture the current application behavior, identify its actual framework and integrations, and document each implemented feature. A source-to-report checklist will prevent the report from describing planned capabilities as existing functions.

13.2 Stronger Route Evaluation

Future development can add or improve weighted shortest-path search, route alternatives, and explicit cost explanations. If A* is used, the heuristic should be matched to the route objective and tested for correctness. Algorithm selection should follow measured needs rather than complexity for its own sake.

13.3 Dynamic Data Integration

If supported by the project's purpose and provider terms, the system could integrate timestamped traffic or incident data. A staged approach should begin with one dependable source, document its limitations, and define fallback behavior before combining multiple sources.

13.4 Personalization and Accessibility

Optional preferences could include avoiding tolls, limiting walking, or selecting a travel mode, provided the underlying data supports them. Preferences should be explicit and reversible. Accessibility improvements include keyboard navigation, clear labels, adequate contrast, and alternatives to map-only communication.

13.5 Evaluation and Deployment

A future evaluation should use a larger set of route scenarios, controlled benchmarks, and—if real travel estimates are offered—observed travel outcomes. Deployment monitoring can track error rates and service latency without collecting unnecessary precise location histories. The team should also document API limits, operating costs, and data licensing.

14. LEARNING OUTCOMES

14.1 Problem Modelling

The project connects a real-world problem to a formal graph model, including vertices, edges, constraints, and route costs. This helps distinguish a shortest-distance problem from a minimum-time or multi-criteria decision problem.

14.2 Algorithm Selection

Reviewing BFS, Dijkstra's algorithm, and A* demonstrates that algorithms differ in their assumptions, optimality guarantees, and search behavior. The correct method depends on whether edges are weighted and whether a suitable heuristic is available.

14.3 Software Design

Separating interface, input validation, route computation, data access, and result validation improves maintainability and makes independent testing possible. Clear request and response contracts also improve error handling.

14.4 Testing and Evidence

The report emphasizes that a visible route is not proof of correctness. Known-answer tests, independent path validation, reproducible benchmarks, and genuine screenshots are needed to substantiate implementation claims.

14.5 Responsible Technology

Commuting tools handle potentially sensitive location information and uncertain estimates. Responsible design requires data minimization, transparent limitations, accessible presentation, and warnings against unsafe interaction while travelling.

15. CONCLUSION

Smart Flow Commute is framed in this report as a web-based intelligent commuting assistant intended to support route planning and clearer journey decisions. The report defines the problem using a graph model, discusses suitable shortest-path algorithms, proposes a modular system architecture, and specifies functional, non-functional, testing, and evaluation requirements.

The key conclusion is that route planning quality depends on more than selecting an algorithm. A useful system also requires a valid road network, meaningful edge costs, reliable location resolution, explicit handling of changing conditions, clear communication of uncertainty, and independent validation. No claim of live traffic integration, AI prediction accuracy, or travel-time savings is made here because supporting implementation evidence and measurements were not supplied.

The recommended next step is to inspect the deployed application and its source, confirm the implemented feature set, execute the defined tests, and insert genuine screenshots and measured results. Completing these tasks will turn this structured project report into an evidence-backed account of the actual Smart Flow Commute implementation.

16. ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty guide, Mrs. Deepa, for her guidance and support throughout the development and documentation of this project. We thank the Department of Computer Science and Engineering, Rajalakshmi Engineering College, for providing the academic environment in which we could explore artificial intelligence concepts and apply them to a practical problem.

We also acknowledge the creators of the software tools, documentation, and educational resources consulted during the project. Finally, we thank our peers and everyone who provided feedback that helped us refine our understanding of intelligent commuting and web application design.

17. REFERENCES

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2021.

[2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[4] R. E. Tarjan, Data Structures and Network Algorithms. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1983.

[5] M. Mitchell, An Introduction to Genetic Algorithms. Cambridge, MA: MIT Press, 1998. (Background on heuristic search; not a claim that this project uses genetic algorithms.)

[6] OpenStreetMap contributors, “OpenStreetMap,” https://www.openstreetmap.org/. Accessed October 23, 2026. Consult the applicable attribution and usage requirements before integration.

[7] Lovable, “AI-powered app builder,” https://lovable.dev/. Accessed October 23, 2026.

[8] Smart Flow Commute project website, https://smart-flow-commute.lovable.app/. Accessed October 23, 2026; detailed deployed behavior should be verified directly before final submission.

[9] Python Software Foundation, “Python Documentation,” https://docs.python.org/3/. Accessed October 23, 2026. Use only if Python is confirmed in the implementation.

[10] Mozilla, “MDN Web Docs,” https://developer.mozilla.org/. Accessed October 23, 2026. General web platform reference.